



NEPAL ENGINEERING COLLEGE
(Affiliated to Pokhara University)
Changunarayan – 04, Bhaktapur, Nepal

Course Project Report on
Face Mask Detection System

Submitted to
**Department of Computer Science
and Engineering**

Submitted by
Bigyan Badal [023-316]

Acknowledgement

I would like to express my sincere gratitude to the Department of Computer Science and Engineering, Nepal Engineering College, for providing me with the opportunity to carry out this individual course project as a part of the Image Processing and Pattern Recognition course.

I am deeply grateful to my course instructor, **Asst. Prof. Hari Mohan Khatri**, for his valuable guidance, suggestions, and continuous support throughout the development of the “Face Mask Detection System Using Deep Learning.” His feedback and encouragement helped me understand and apply the concepts of image processing, computer vision, deep learning, and pattern recognition effectively in this project.

Abstract

This project presents a Face Mask Detection System Using Deep Learning to automatically identify whether a person is wearing a face mask or not. A Convolutional Neural Network (CNN) model was developed and trained specifically for this project using a prepared face-mask image dataset. The model was trained using TensorFlow and Keras through a Python-based training module. MediaPipe is used for detecting faces from input images and real-time webcam frames, while OpenCV is used for image processing and camera handling. The trained CNN classifies detected faces into two categories: With Mask and Without Mask.

The system provides two main detection modes: image-based detection, where users can upload an image, and real-time webcam detection for continuous monitoring. A Streamlit-based user interface is developed to provide a simple and interactive platform for accessing the system. After detecting a face, the system extracts the face region, preprocesses the image, and passes it to the trained CNN model for classification. The predicted class and confidence score are displayed along with a bounding box around the detected face. The project demonstrates the practical application of image processing, computer vision, deep learning, and pattern recognition techniques to solve a real-world problem. The developed system can be useful for automated mask monitoring in public places, educational institutions, offices, and healthcare environments. Overall, the project provides a complete workflow from dataset preparation and model training to face detection, mask classification, and user-friendly application deployment.

Keywords: Face Mask Detection, Deep Learning, Convolutional Neural Network (CNN), Computer Vision, MediaPipe, OpenCV, TensorFlow, Keras, Image Processing, Real-Time Detection, Face Detection, Streamlit.

Table of Contents

Acknowledgement	i
Abstract.....	ii
Table of Contents	iii
List of Figures.....	v
List of Tables.....	vi
Chapter 1: Introduction.....	1
1.1 Background and Motivation	1
1.2 Problem statement.....	1
1.3 Aims and Objectives	2
1.3.1 Aims	2
1.3.2 Objectives	2
1.4 Scope and Application	2
1.5 Feasibility Study	3
1.5.1 Economic Feasibility	3
1.5.2 Technical Feasibility	3
1.5.3 Operational Feasibility.....	3
Chapter 2: Literature Review	4
2.1 Introduction to Face Mask Detection.....	4
2.2 Image Processing and Computer Vision	4
2.3 Face Detection Techniques	5
2.4 Deep Learning and Image Classification.....	5
2.5 Convolutional Neural Network (CNN).....	6
2.6 Dataset and Data Preprocessing.....	7
2.7 Model Training and Classification.....	7
2.8 Real-Time Face Mask Detection.....	8
Chapter 3: System Analysis and Design.....	9
3.1 Requirement Analysis	9
3.1.1 Hardware Requirements.....	9
3.1.2 Software Requirements	9
3.2 Dataset Description	10
3.3 Flowchart	11

3.4 Archirecture Diagram.....	12
Chapter 4: Implementation.....	13
4.1 Technologies Used	13
4.2 Modules.....	13
4.2.1 Dataset and Preprocessing Module	14
4.2.2 CNN Model Training Module.....	14
4.2.3 Face Detection Module	14
4.2.4 Mask Classification Module	14
4.2.5 Image Detection Module.....	14
4.2.6 Real-Time Webcam Detection Module.....	15
4.2.7 Streamlit Application Module.....	15
4.3 Algorithms Used	15
4.3.1 Convolutional Neural Network (CNN).....	15
4.3.2 MediaPipe Face Detection	15
4.3.3 Adam Optimization Algorithm	15
4.3.4 Binary Classification Using Sigmoid.....	16
Chapter 5: Testing.....	17
5.1 Test Cases.....	17
5.2 Accuracy and Performance Evaluation	18
5.3 Results.....	18
5.3.1 Test Dataset Class Distribution	18
5.3.2 Training and Validation Performance	19
5.3.3 Confusion Matrix	20
5.3.5 Overall Results.....	20
Chapter 6: Conclusion	21
6.1 Achievements	21
6.2 Limitations	21
6.3 Expected Output.....	22
6.4 Future Works	22
References	23

List of Figures

Figure 1: Flowchart Of System.....	11
Figure 2:Architecture Diagram	12
Figure 3:Test Dataset Class Distribution	19
Figure 4:Training and Validation Performance	19
Figure 5:Confusion Matrix	20
Figure 6:Expected Output	22

List of Tables

Table 1:Hardware Requirements.....	9
Table 2:Software Requirements.....	10
Table 3:Technologies Used	13
Table 4:Test Cases.....	17
Table 5: Accuracy and Performance Evaluation	18

Chapter 1: Introduction

1.1 Background and Motivation

Face masks are an effective preventive measure for reducing the transmission of airborne diseases, especially in crowded and public environments. During health emergencies, ensuring proper mask usage becomes important for maintaining public safety. However, manually monitoring mask usage among a large number of people can be difficult, time-consuming, and inefficient. The development of image processing and computer vision technologies has made it possible to automate such monitoring tasks. Deep learning, particularly Convolutional Neural Networks (CNNs), has shown strong capabilities in image classification and pattern recognition. Motivated by these developments, this project focuses on developing an automated Face Mask Detection System using deep learning. A CNN model is developed and trained using a prepared dataset containing images of faces with and without masks. MediaPipe is used to detect faces, while OpenCV supports image processing and real-time webcam handling. The system classifies detected faces into With Mask and Without Mask categories and displays the prediction with a confidence score. A Streamlit-based interface is also developed to make the system simple and interactive for users. The motivation behind this project is to demonstrate how deep learning and computer vision can be applied to develop a practical and automated solution for face-mask monitoring.

1.2 Problem statement

Monitoring whether individuals are wearing face masks is a challenging task, particularly in crowded public environments where a large number of people need to be observed simultaneously. Manual monitoring requires continuous human attention, making the process time-consuming, inefficient, and difficult to maintain consistently. In addition, human monitoring may not provide reliable results when the number of individuals increases. Therefore, an automated system is required to detect faces and determine whether individuals are wearing face masks. The main problem addressed by this project

is the development of an automated face mask detection system that can identify faces from images and real-time webcam input and classify them into With Mask and Without Mask categories. The system is designed to use image processing, face detection, and deep learning techniques to perform the classification process. The system should also provide clear prediction results through a simple and user-friendly interface. This project addresses these requirements by developing and training a CNN model using a prepared face-mask dataset and integrating the model with MediaPipe, OpenCV, and a Streamlit-based application.

1.3 Aims and Objectives

1.3.1 Aims

The main aim of this project is to develop an automated face mask detection system using a CNN model trained on a prepared face-mask dataset. The system aims to detect faces from images and real-time webcam input and accurately classify them as With Mask or Without Mask.

1.3.2 Objectives

- To develop and train a CNN model using a prepared face-mask dataset for classifying faces into With Mask and Without Mask categories.
- To implement a user-friendly system that detects faces from images and real-time webcam input and displays the classification results.

1.4 Scope and Application

The scope of this project is to develop an automated face mask detection system that can identify faces and classify them as With Mask or Without Mask. The system supports both image-based detection and real-time detection through a webcam. It can be applied in public places, educational institutions, offices, healthcare facilities, and other environments where automated face mask monitoring may be required. The system can also serve as a foundation for further development of real-time surveillance and computer vision-based safety applications.

1.5 Feasibility Study

1.5.1 Economic Feasibility

The proposed system is economically feasible because it can be developed using freely available and open-source software tools such as Python, TensorFlow, Keras, OpenCV, MediaPipe, and Streamlit. The system does not require expensive hardware or specialized equipment, as it can operate on a standard computer with a webcam. Therefore, the overall development and implementation cost of the system is relatively low.

1.5.2 Technical Feasibility

The system is technically feasible because the required technologies and development tools are readily available and compatible with the proposed implementation. Python is used for model development and application programming, while TensorFlow and Keras are used to develop and train the CNN model. MediaPipe and OpenCV support face detection and image processing, and Streamlit is used to develop the user interface. The system can be implemented and executed on a standard computer with appropriate software dependencies.

1.5.3 Operational Feasibility

The proposed system is operationally feasible because it provides a simple and user-friendly interface for face mask detection. Users can upload an image or use a webcam to perform real-time detection without requiring extensive technical knowledge. The system displays the detected face along with the predicted mask status and confidence score, making the results easy to understand. Therefore, the system can be practically used in environments such as educational institutions, offices, healthcare facilities, and public places.

Chapter 2: Literature Review

2.1 Introduction to Face Mask Detection

Face mask detection is a computer vision-based application that identifies whether a person is wearing a face mask in an image or video. The increasing need for automated monitoring during public health emergencies led to significant research in this area. Deep learning and computer vision techniques have been widely explored for detecting and classifying masked faces, with CNN-based approaches being among the commonly studied methods.

In a face mask detection system, the process generally involves detecting a face from an input image or video frame and then determining whether the detected face belongs to a masked or unmasked category. This approach combines face detection with image classification to produce the final result. Research has also explored different datasets, detection methods, and deep learning architectures to improve the accuracy and practical performance of such systems.

In this project, face mask detection is implemented by first detecting the face using MediaPipe and then classifying the detected face using a CNN model developed and trained specifically for this project. The system supports both image-based and real-time webcam-based detection, making it suitable for practical face mask monitoring applications.

2.2 Image Processing and Computer Vision

Image processing involves applying various techniques to digital images to improve, transform, and prepare them for further analysis. It includes operations such as image resizing, normalization, and preprocessing, which are important for preparing input data for computer vision and deep learning models. Computer vision is a field of artificial intelligence that enables computers to extract meaningful information from images and videos. It is widely used in applications such as face detection, image classification, object recognition, and real-time monitoring.

In this project, image processing and computer vision techniques are used to process uploaded images and real-time webcam frames. OpenCV is used for image handling, preprocessing, and webcam input, while MediaPipe is used to detect faces from the input frames. The detected face region is then prepared and provided to the CNN model developed and trained specifically for this project. Thus, image processing and computer vision form the foundation of the face detection and mask classification process in the proposed system.

2.3 Face Detection Techniques

Face detection is an essential computer vision process used to identify and locate human faces within an image or video frame. In the proposed system, face detection is required before mask classification because the CNN model needs the detected facial region as its input. Accurate face localization helps isolate the relevant facial area from the background and allows the subsequent classification process to focus on the required region. The system supports face detection for both uploaded images and real-time webcam frames.

In this project, MediaPipe Face Detection is used to detect human faces from the input images and webcam frames. It identifies the facial region and provides a bounding box around the detected face, which is then used for further processing. MediaPipe is particularly suitable for this project because it supports efficient and real-time face detection, which is important for webcam-based applications. After the face is detected, the corresponding region is processed and passed to the CNN model developed and trained specifically for this project. The CNN then classifies the detected face into With Mask or Without Mask categories. Thus, MediaPipe performs the face localization stage, while the custom-trained CNN performs the mask classification stage of the proposed system.

2.4 Deep Learning and Image Classification

Deep learning is used in this project to automatically learn the visual features required for face mask classification. A Convolutional Neural Network (CNN) model is developed and trained using the prepared face-mask dataset so that it can distinguish between masked and

unmasked faces. During training, the model learns the relevant patterns from the training images and uses these learned patterns to make predictions on new face images.

In the proposed system, the detected face region from MediaPipe is preprocessed and provided as input to the trained CNN model. The model then performs image classification and predicts whether the detected face belongs to the With Mask or Without Mask category. Therefore, deep learning is mainly responsible for the mask classification stage of the system, while MediaPipe handles face detection and OpenCV supports image and webcam processing.

2.5 Convolutional Neural Network (CNN)

A Convolutional Neural Network (CNN) is used in this project as the main classification model for distinguishing between faces with masks and faces without masks. The CNN learns image features directly from the training images by applying convolution operations to different regions of an image. In the initial layers, the model learns simple features such as edges, shapes, and patterns, while deeper layers combine these features to identify more meaningful visual characteristics. Through the training process, the model adjusts its internal parameters based on the patterns present in the prepared face-mask dataset. As a result, the trained model learns the visual differences between masked and unmasked faces and can use these learned features when classifying new input images.

CNN is selected for this project because face mask classification is an image-based problem in which the visual appearance of the face changes depending on whether a mask is present. CNN can automatically learn relevant spatial features from images and use them for classification without requiring manual feature extraction. In the proposed system, the face detected by MediaPipe is preprocessed and provided to the CNN as input. The trained CNN then analyzes the visual features of the detected face and predicts either With Mask or Without Mask along with a confidence score. Therefore, CNN performs the core classification task of the system and enables the project to automatically distinguish between masked and unmasked faces.

2.6 Dataset and Data Preprocessing

A dataset is a collection of images or data samples used to train and evaluate a machine learning model. In this project, a prepared face-mask dataset is used, which contains images belonging to two classes: With Mask and Without Mask. The dataset is organized into separate training, validation, and testing sets. The training set is used by the CNN model to learn the visual patterns of masked and unmasked faces, the validation set is used to monitor the model during training, and the testing set is used to evaluate the model on previously unseen images.

Before training, the images are preprocessed to provide a consistent input format for the CNN model. Each image is resized to 128×128 pixels, ensuring that all images have the same dimensions. Pixel values are normalized by scaling them from the original range of 0–255 to a range of 0–1. Data augmentation is also applied during training using techniques such as rotation, zooming, and horizontal flipping. These transformations provide additional variations of the training images and help the model learn more general visual patterns. The preprocessed and augmented images are then provided to the CNN model through the training process, allowing the model to learn the characteristics required to distinguish between With Mask and Without Mask faces.

2.7 Model Training and Classification

The CNN model is trained using the preprocessed face-mask dataset to learn the patterns of With Mask and Without Mask images. The model is trained in batches using a loss function and the Adam optimizer to improve its classification performance over successive epochs.

After training, the CNN receives the detected and preprocessed face as input and classifies it as With Mask or Without Mask. The trained model is saved as `mask_detector.keras` and used by the detection and Streamlit application to display the predicted mask status and confidence score.

2.8 Real-Time Face Mask Detection

Real-time face mask detection involves continuously processing video frames captured through a webcam to detect faces and determine their mask status. It allows the system to perform continuous detection instead of processing only a single image.

In this project, OpenCV captures webcam frames, MediaPipe detects the face, and the detected face region is passed to the trained CNN model for classification as With Mask or Without Mask. The prediction is displayed on the video frame along with the bounding box and confidence score.

Chapter 3: System Analysis and Design

3.1 Requirement Analysis

Requirement analysis identifies the hardware and software resources required to develop, train, test, and operate the proposed face mask detection system. The requirements are defined based on the actual implementation of the system using Python, CNN, TensorFlow, OpenCV, MediaPipe, and Streamlit.

3.1.1 Hardware Requirements

The hardware requirements specify the minimum physical components needed to run the Face Mask Detection System effectively. A processor such as Intel Core i3 or above and at least 8 GB of RAM are recommended for smooth model execution and real-time detection. The system requires a minimum of 10 GB of storage along with a keyboard, mouse, and webcam for user input and real-time face detection. A monitor or display is also required to view the application interface and detection results.

Table 1:Hardware Requirements

S.N.	Hardware Component	Minimum Requirement
1	Processor	Intel Core i3 or above
2	RAM	8 GB or above
3	Storage	10 GB or above
4	Input Device	Keyboard and Mouse
5	Camera	Webcam for real-time detection
6	Output Device	Monitor or Display

3.1.2 Software Requirements

The software and technology requirements define the tools and frameworks needed to develop and run the Face Mask Detection System. Python is used as the main programming language, while TensorFlow and Keras are used for building and training the deep learning model. OpenCV and MediaPipe support image processing and real-time face detection,

while Streamlit provides the web-based user interface. The system is developed using Visual Studio Code on Windows, with NumPy used for numerical and array-based computations.

Table 2:Software Requirements

S.N.	Software / Technology	Requirement
1	Operating System	Windows
2	Programming Language	Python
3	Development Environment	Visual Studio Code
4	Deep Learning Framework	TensorFlow
5	Deep Learning API	Keras
6	Computer Vision Library	OpenCV
7	Face Detection Framework	MediaPipe
8	Web Application Framework	Streamlit
9	Numerical Processing	NumPy

3.2 Dataset Description

The prepared face-mask dataset used in this project consists of facial images categorized into two classes: With Mask and Without Mask. The dataset is organized into separate train, val, and test folders. The train and val datasets are used during the model development process, while the test dataset is used for evaluating the final classification performance.

Before training, the images are resized to 128×128 pixels and normalized by scaling the pixel values using a factor of $1/255$. Data augmentation is applied to the training images using rotation, zoom, and horizontal flipping to provide variations in the training data. The train and val images are loaded using `flow_from_directory()` with binary class mode for the two mask categories.

The dataset structure contains separate train, val, and test directories, with each directory containing With_mask and Without_mask subdirectories. This organization allows the images to be properly categorized and used for the respective stages of the face mask detection system.

3.3 Flowchart

The flowchart represents the overall workflow of the developed face mask detection system. The process begins with the user providing an input through an image or real-time webcam. The system then detects the face using MediaPipe Face Detection. After detecting the face, the facial region is extracted and preprocessed by resizing and normalization. The processed face is then passed to the CNN model trained for mask classification. The model classifies the detected face as With Mask or Without Mask and generates the corresponding confidence score. Finally, the prediction and detection result are displayed to the user through the application.

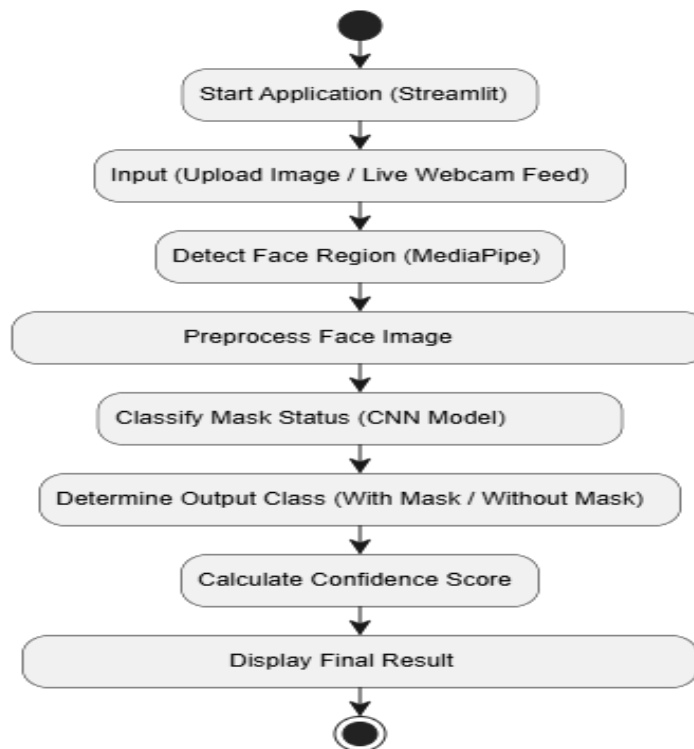


Figure 1: Flowchart Of System

3.4 Archirecture Diagram

The system architecture shows the complete structure of the Face Mask Detection System. The user provides an image or live webcam feed through the Streamlit interface. OpenCV and MediaPipe detect and crop the face, followed by preprocessing of the input image. The preprocessed face is passed to the CNN model stored in `mask_detector.keras`, which classifies the face as With Mask or Without Mask. Finally, the predicted class and confidence score are sent to the output visualization module, where the result is displayed with a bounding box, label, and confidence score.

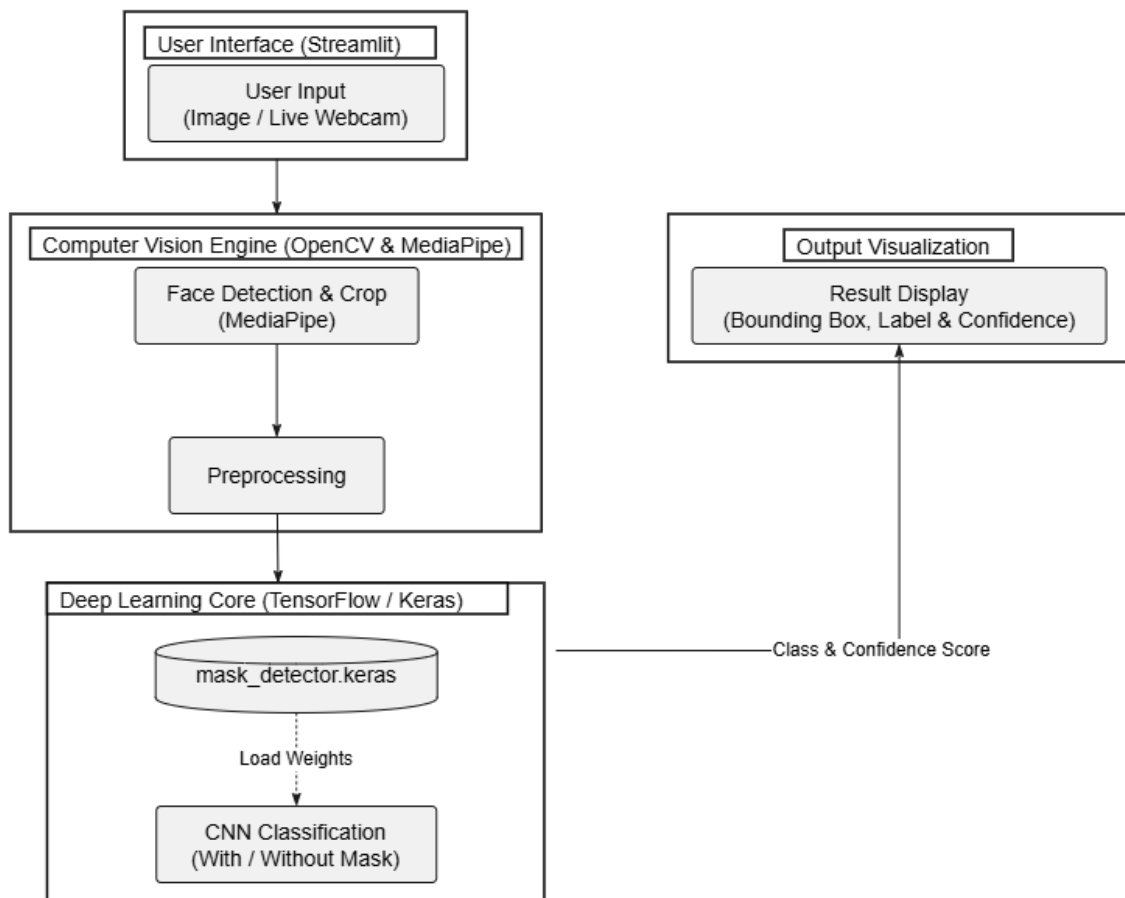


Figure 2:Architecture Diagram

Chapter 4: Implementation

4.1 Technologies Used

The face mask detection system was implemented using a combination of programming tools, deep learning frameworks, and computer vision libraries. Python was used as the main programming language, while TensorFlow and Keras were used to develop and train the CNN model. OpenCV was used for image processing and webcam-based video capture, and MediaPipe was used for face detection. NumPy was used for numerical and image-array operations, Matplotlib was used to visualize the training results, and Streamlit was used to develop the interactive web application.

Table 3:Technologies Used

S.N.	Technology	Purpose
1	Python	Main programming language
2	TensorFlow	CNN model development and training
3	Keras	Building and training the CNN model
4	OpenCV	Image processing and webcam capture
5	MediaPipe	Face detection
6	NumPy	Numerical and image-array processing
7	Matplotlib	Training result visualization
8	Streamlit	Interactive web application development

4.2 Modules

The developed face mask detection system is organized into several modules, with each module performing a specific task in the overall system. The modules are implemented according to the actual workflow of the developed application.

4.2.1 Dataset and Preprocessing Module

This module loads the prepared face-mask dataset and prepares the images for model training. The images are resized to 128×128 pixels and normalized by scaling their pixel values. Data augmentation is applied to the training images using rotation, zoom, and horizontal flipping.

4.2.2 CNN Model Training Module

This module builds and trains the CNN model using the preprocessed training data. The model consists of convolutional layers, max-pooling layers, a flatten layer, dense layers, and dropout. The model uses the Adam optimizer and binary cross-entropy loss to learn the two mask classes.

4.2.3 Face Detection Module

This module uses MediaPipe Face Detection to identify faces in input images and video frames. The detected face coordinates are obtained from the bounding box and used to extract the corresponding face region for further classification.

4.2.4 Mask Classification Module

The extracted face region is resized and normalized before being passed to the trained CNN model. The model predicts whether the detected face belongs to the With Mask or Without Mask category and calculates the corresponding confidence score.

4.2.5 Image Detection Module

This module allows users to upload an image through the Streamlit application. The uploaded image is processed for face detection and mask classification, and the resulting image is displayed with the detected face, mask status, and confidence score.

4.2.6 Real-Time Webcam Detection Module

This module captures live video frames using OpenCV and processes them continuously for face detection and mask classification. The prediction is displayed on the video frame using a bounding box, mask label, and confidence score.

4.2.7 Streamlit Application Module

This module provides the user interface for interacting with the system. The application includes Home, Image Detection, and Webcam Detection options, allowing users to perform face mask detection through a simple web-based interface.

4.3 Algorithms Used

The developed face mask detection system uses the following four main algorithms and techniques:

4.3.1 Convolutional Neural Network (CNN)

The CNN is the main classification algorithm used in the system. It learns visual features from the prepared face-mask images and classifies detected faces into With Mask and Without Mask categories. The implemented model consists of convolutional, max-pooling, flatten, dense, and dropout layers.

4.3.2 MediaPipe Face Detection

MediaPipe Face Detection is used to detect and locate faces in input images and real-time video frames. The detected face region is extracted using its bounding-box coordinates and then passed to the CNN model for mask classification.

4.3.3 Adam Optimization Algorithm

The Adam optimizer is used during CNN training to update the model parameters and reduce the classification error. In the implemented model, Adam is configured with a learning rate of 0.0001.

4.3.4 Binary Classification Using Sigmoid

The system performs binary classification because it has two classes: With Mask and Without Mask. A sigmoid activation function is used in the final layer of the CNN to produce the prediction value, which is then used to determine the corresponding mask category.

Chapter 5: Testing

5.1 Test Cases

The developed face mask detection system was tested to verify the functionality of its major components. The testing was performed using both image input and real-time webcam input. The system was evaluated for face detection, mask classification, and confidence score display.

Table 4:Test Cases

Test Case	Test Input	Expected Output	Actual Output	Status
TC-01	Image with mask	Face detected and classified as With Mask	With Mask detected	Pass
TC-02	Image without mask	Face detected and classified as Without Mask	Without Mask detected	Pass
TC-03	Webcam with mask	Face detected and classified as With Mask	With Mask detected	Pass
TC-04	Webcam without mask	Face detected and classified as Without Mask	Without Mask detected	Pass
TC-05	Image or webcam frame containing a face	Face detected with bounding box	Face detected with bounding box	Pass
TC-06	Detected face	Prediction confidence displayed	Confidence score displayed	Pass

The test results show that the major functional components of the proposed system performed as expected. The system was able to detect faces from image and webcam inputs and classify the detected faces into With Mask and Without Mask categories. It also displayed the corresponding confidence score for the prediction.

5.2 Accuracy and Performance Evaluation

The performance of the trained CNN model was evaluated using the test dataset containing 1,809 images. The evaluation was performed using accuracy, precision, recall, and F1-score to measure how effectively the model classified the With Mask and Without Mask categories.

Table 5: Accuracy and Performance Evaluation

S.N.	Evaluation Metric	With Mask	Without Mask
1	Precision	0.98	0.98
2	Recall	0.98	0.98
3	F1-Score	0.98	0.98

The model achieved an overall accuracy of approximately 98.23% on the test dataset. The similar precision, recall, and F1-score values for both classes indicate that the model provided consistent classification performance for With Mask and Without Mask images.

5.3 Results

The developed face mask detection system was evaluated using the test dataset and real-time inputs. The CNN model achieved an overall accuracy of approximately 98.23%, demonstrating effective classification performance for both With Mask and Without Mask categories.

5.3.1 Test Dataset Class Distribution

The test dataset consists of 1,809 images, including 849 images in the With Mask class and 960 images in the Without Mask class. This distribution represents the data used to evaluate the classification performance of the trained CNN model.

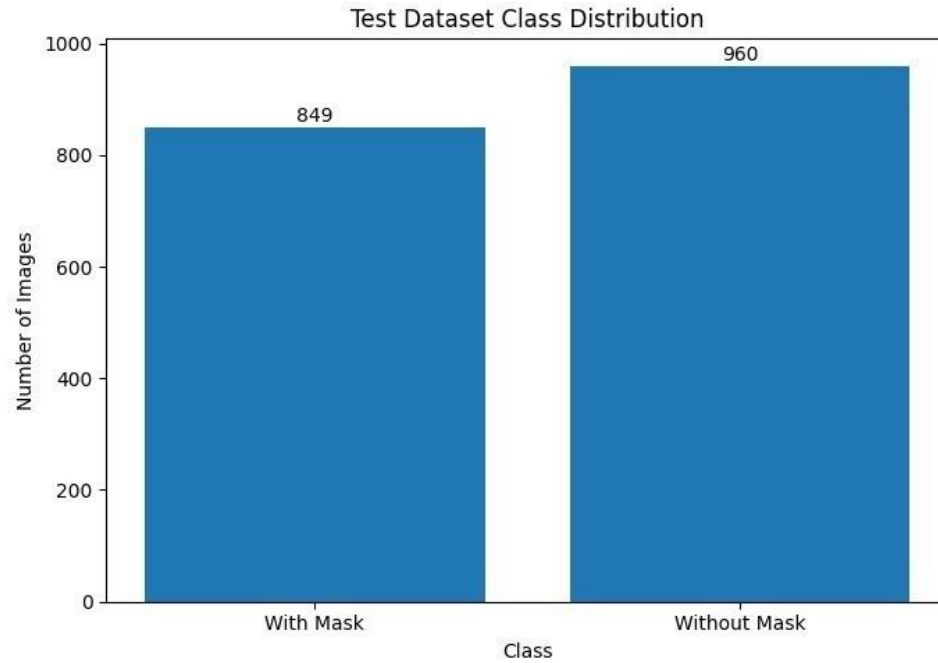


Figure 3:Test Dataset Class Distribution

5.3.2 Training and Validation Performance

The training and validation accuracy and loss curves show the learning performance of the CNN model throughout the training process. The curves provide an overview of how the model's performance changes across the training epochs.

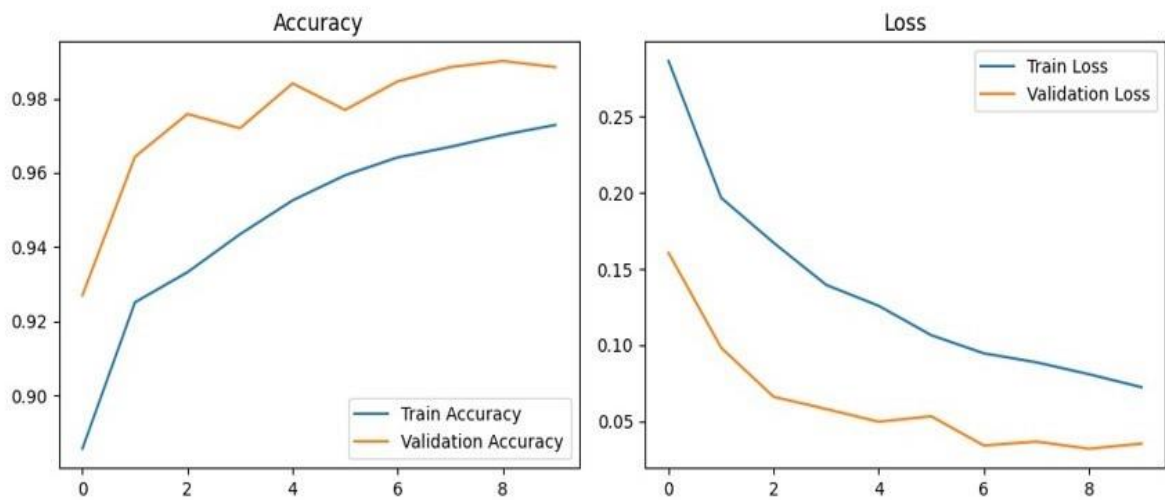


Figure 4:Training and Validation Performance

5.3.3 Confusion Matrix

The confusion matrix provides a detailed representation of the model's classification results. The model correctly classified 833 With Mask images and 945 Without Mask images, while 16 With Mask images and 15 Without Mask images were incorrectly classified.

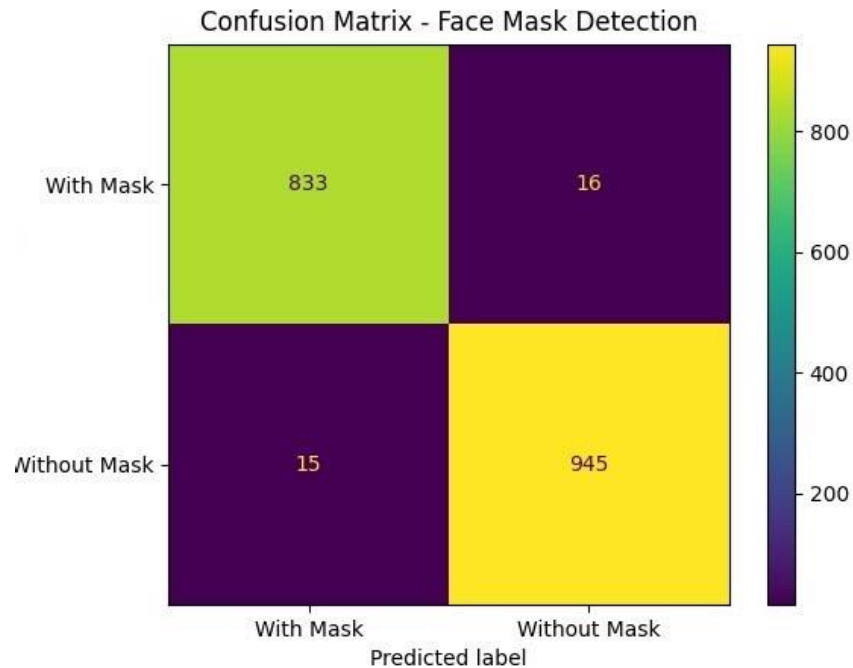


Figure 5:Confusion Matrix

5.3.5 Overall Results

The developed face mask detection system achieved an overall accuracy of approximately 98.23% on the test dataset. The results from the confusion matrix and classification metrics indicate that the trained CNN model can effectively distinguish between With Mask and Without Mask faces. The system also successfully performs face detection and classification using real-time webcam input.

Chapter 6: Conclusion

6.1 Achievements

The project successfully developed a face mask detection system capable of detecting human faces and classifying them into With Mask and Without Mask categories. A CNN model was developed and trained using the prepared face-mask dataset, rather than using a pretrained classification model. MediaPipe was integrated for face detection, while OpenCV was used for image and real-time webcam processing.

The developed system achieved approximately 98.23% accuracy on the test dataset, with precision, recall, and F1-score of approximately 0.98 for both classes. The system was also successfully tested with image inputs and real-time webcam inputs. The trained model was integrated into the application to provide mask status and confidence score, demonstrating the successful implementation of the proposed face mask detection system.

6.2 Limitations

Although the developed face mask detection system provides good classification performance, it has some limitations. The system primarily focuses on detecting a single mask status for detected faces and its performance may vary under challenging conditions such as poor lighting, extreme face angles, partial face occlusion, or unclear camera input. The accuracy of real-time detection also depends on the quality and resolution of the webcam.

Another limitation is that the CNN model is trained specifically on the prepared face-mask dataset, so its performance may vary when it encounters images with facial appearances, mask types, or environmental conditions that are significantly different from the training data. Additionally, the current system is designed mainly for With Mask and Without Mask classification and does not identify other conditions such as incorrectly worn masks.

6.3 Expected Output



Figure 6:Expected Output

6.4 Future Works

The developed face mask detection system can be further improved and extended in several ways. Future work may include:

1. The system can be extended to detect incorrectly worn masks, such as masks covering only the mouth or only the chin.
2. The model can be improved by training with a larger and more diverse dataset containing different lighting conditions, face angles, mask types, and environments.
3. The system can be optimized for faster and more efficient real-time detection on low-resource devices.
4. The application can be extended to support multiple-face detection and classification in crowded environments.
5. Future versions can be deployed on mobile or embedded devices for practical real-world applications.

References

- [1] M. Roomi, M. Mansoor, M. Beham, and M. Parisa, “A review of face recognition methods,” *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 27, no. 4, p. 1356005, 2022.
- [2] S. Syed Navaz, T. Dhevi Sri, and P. Mazumder, “Face recognition using principal component analysis and neural network,” *International Journal of Computer Networking, Wireless and Mobile Communications (IJCNWMC)*, vol. 3, pp. 245–256, Mar. 2013.
- [3] M. A. Turk and A. P. Pentland, “Face recognition using eigenfaces,” in *Proc. IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 1991, pp. 586–591.
- [4] H. Li, Z. Lin, X. Shen, J. Brandt, and G. Hua, “A convolutional neural network cascade for face detection,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 5325–5334.
- [5] W. Bu, J. Xiao, C. Zhou, M. Yang, and C. Peng, “A cascade framework for masked face detection,” in *Proc. IEEE International Conference on Cyberspace and Information Technology (CIS-RAM)*, Ningbo, China, 2017, pp. 458–462.
- [6] S. Ge, J. Li, Q. Ye, and Z. Luo, “Detecting masked faces in the wild with LLE-CNNs,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2682–2690.
- [7] G. Opitz, G. Waltner, G. Poier, et al., “Grid loss: Detecting occluded faces,” in *Proc. European Conference on Computer Vision (ECCV)*, 2016, pp. 386–402.
- [8] X. Zhu and D. Ramanan, “Face detection, pose estimation, and landmark localization in the wild,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 2879–2886.
- [9] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A unified embedding for face recognition and clustering,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823.
- [10] A. Bansal, R. Ranjan, C. D. Castillo, and R. Chellappa, “Deep features for recognizing disguised faces in the wild,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020.

